

Juniper

Property-Directed Mathematics with Explicit Obligation Frontiers

A conservative extension of Lean’s authoring discipline,
with behavioral contracts and proof-producing computation

The central idea. A mathematical step should say what the author intends to do. Juniper should infer the evidence that licenses that step, and expose a finite, proof-backed account of what remains missing. It must never obtain concision by silently changing the object, the hypotheses, or the claim.

This article reads the two round-three syntheses together, checks representative source declarations in ProveIt, the FLT repository, and Leant, and develops a specific fourth-round increment: *residual proof telescopes and a bounded antichain algorithm for alternative sufficient premise sets*.

The accompanying reference implementation parses a small ground propositional language, computes these frontiers, validates derivation graphs, and exports generic Lean implication proofs. The larger mathematical surface and the Lean integration remain designs, not implemented features.

Prepared for Vladimir Reshetnikov

September 6, 2026

Evidence status: paper proofs; executed Python reference and tests; uncompiled generated Lean. No productivity study or full repository build is claimed.

Abstract

Lean can already express remarkably rich mathematical specifications. The difficulty is often that their consequences must be repeatedly reconstructed at use sites: an identity on an open set becomes a neighborhood identity; a smoothness hypothesis licenses a chain rule at a point in a function's image; an integer quotient agrees with a rational quotient only after divisibility is established. Merely making tactic scripts less visible does not solve this interface problem.

Juniper is a proposed mathematical authoring layer in which types describe an object's carrier, its fixed structures, its established properties, and the behavior expected of operations. The extension is primarily to elaboration: a proof-backed property index and property-implicit arguments compile to ordinary Lean terms. There is no semantic subtyping inside definitional equality, no automatic change of structure, and no new logical axiom. A richer kernel refinement former is evaluated as a research option, not presumed necessary.

The distinctive proposal is to make unfinished reasoning precise. A step produces a residual telescope and a proof valid under exactly that telescope. In a finite grounded Horn fragment, an antichain algorithm computes every inclusion-minimal set of designated additional premises sufficient for the goal. The article proves soundness, termination, relative completeness, and schedule independence for the specified algorithm, explains the exponential worst case, and specifies a bounded grounding discipline. These frontiers are explanations and proof plans, not permissions to assume their leaves.

Worked studies cover the autonomous-derivative proof, non-vacuous Frey exact division, analytic inverse derivatives certified through differential polynomials, quotient descent, and behavior-directed synthesis. The companion passes 32 unit tests; 400 generated rule profiles agree with an independent exhaustive oracle on 2,800 goal frontiers and with reversed rule schedules. It also checks 189 admissible arithmetic fixtures and computes inverse-derivative polynomials through order six. These are finite implementation checks, not a formal verification of the implementation. The generated Lean file was not compiled in this environment.

Contents

Abstract	1
1 The proposal in one page	4
2 What the sources actually establish	4
2.1 The two syntheses should be read as a reconciled discussion	4
2.2 Representative code, not a caricature of Lean	5
2.3 Existing Lean features are baselines, not Juniper inventions	6
3 What a mathematician would write	6
3.1 State the mathematical move, retain its scope	6
3.2 Contracts at call sites	7
3.3 An incomplete step should remain visibly incomplete	7
3.4 Construction is a separate act	8
4 A property-aware typing discipline	8
4.1 Three environments, not one bag of hints	8
4.2 Local rows and exported types	9
4.3 Property-implicit arguments	9
4.4 Proof irrelevance and dependent transport	9
5 Residual telescopes: the meaning of an unfinished proof	10
5.1 From a missing premise to a conditional derivation	10
5.2 Why a telescope is necessary	10

5.3	Residual composition	10
5.4	Drafts, assertions, and publication	11
6	A bounded algorithm for alternative obligations	11
6.1	The finite problem	11
6.2	Antichain propagation	11
6.3	Soundness and termination	12
6.4	Relative completeness and schedule independence	13
6.5	A checkable completeness certificate	13
6.6	Example and cost	14
7	Grounding rules without unbounded search	14
7.1	A finite automatic fragment must say how it becomes finite	14
7.2	Two completeness flags	14
7.3	Demand-directed closure	15
7.4	Scope and rewriting	15
8	Behavioral types beyond unary predicates	15
8.1	Total functions and restricted-domain functions differ	15
8.2	Variance with a proof, not a slogan	16
8.3	Whole-function and relational contracts	16
8.4	Behavioral abstractions and negative evidence	17
9	Case study: exact arithmetic in the Frey model	17
9.1	The source already contains a property-rich object	17
9.2	Extract a satisfiable arithmetic interface	17
9.3	Exact quotient evidence	18
9.4	A compact authoring sketch and its frontier	18
10	Case study: analytic inverse derivatives with certified algebra	19
10.1	A full mathematical contract	19
10.2	A differential-polynomial recurrence	19
10.3	The first formulas and checkable invariants	20
10.4	How Juniper would factor the work	21
10.5	Negative neighbors	21
11	Case study: descent through a quotient	22
11.1	An operation with a universal property	22
11.2	A property-directed construction	22
11.3	Useful failure cases	23
12	A hybrid proof assistant and computer algebra system	23
12.1	A computation should return a typed mathematical result	23
12.2	Separate algebraic normalization from analytic licensing	23
12.3	Locality, precision, and measure are contract parameters	23
12.4	Certificate provenance and result strength	24
13	Integrating Leant without weakening its boundaries	24
13.1	Use existing candidate identity infrastructure	24
13.2	A proof-carrying synthesis request	25
13.3	Requirements can guide candidate generation	25
13.4	Positive, negative, and bounded evidence	25
14	How far should Lean’s type system be extended?	25
14.1	Recommended first layer: an elaboration extension	25
14.2	An optional native refinement former	26
14.3	What should not enter definitional equality	26
14.4	Proof obligations for a kernel experiment	26
15	An implementation architecture with explicit acceptance criteria	27

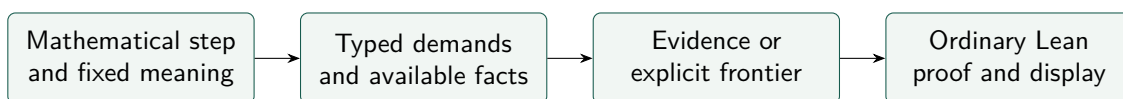
15.1 Seven components and one trusted endpoint	27
15.2 Three forms of fidelity	27
15.3 Avoiding a theorem-backed but misleading narrative	28
15.4 A relative elaboration theorem	28
16 The delivered executable slice and what was tested	28
16.1 An implemented language small enough to describe completely	28
16.2 Derivation validation and export	29
16.3 Executed results	29
16.4 The exponential family is visible even in a tiny example	30
16.5 Evidence ledger	31
17 Evaluation and a falsifiable development program	31
17.1 Separate language gains from library gains	31
17.2 Concrete acceptance milestones	31
17.3 Adversarial acceptance matrix	32
17.4 Answers to the syntheses' remaining questions	32
18 Conclusion	33
A Reproduction and artifact contents	33
B Further inverse-derivative polynomial output	34
C Source scope and attribution	35
References	35

1 The proposal in one page

Juniper should extend the type system that an author interacts with, while initially leaving Lean’s trusted type theory unchanged. Its basic judgment is not simply “this expression has type A ,” but “this exact expression has carrier A , under these fixed structures, with these proved properties, and this use demands the following evidence.”

The object remains ordinary Lean data. A local assertion enriches an evidence index, not the object’s identity. At an exported interface, the same information can be packaged in a subtype or a structure. A behavioral annotation is an ordinary proposition about a function, possibly involving several inputs, several outputs, other functions, measures, domains, or precision indices. It is not an unchecked label.

Juniper’s authoring loop is:



Three choices distinguish this proposal from a restatement of round three.

1. **Unfinished steps have a mathematical semantics.** Their output is a residual proof telescope, rather than a cache entry that looks like a proved fact. Dependent obligations and witness choices remain ordered.
2. **Alternative missing-premise explanations have an algorithm.** For an explicitly finite fragment, Juniper computes an antichain of sufficient premise sets and retains an acyclic derivation for each route. The advertised completeness is relative to that fragment and its designated premises, never to all of mathematics.
3. **The design is accompanied by a narrow executable slice.** A small parser, solver, derivation validator, and generic Lean exporter exercise the algorithm and failure states. They do not implement Lean’s elaborator or pretend that symbolic atom names are mathematical theorems.

The appropriate stopping condition is equally important. If the same improvement is obtained from a property library, existing tactics, and a better renderer, those should be the product. Juniper is justified only by additional, measured improvements in authoring, diagnosis, or maintenance.

2 What the sources actually establish

2.1 The two syntheses should be read as a reconciled discussion

From Properties to Proof Obligations and *Nine Refinements* converge on the same architecture: fixed objects, locally accumulated evidence, theorem-backed behavioral contracts, structure selection before property inference, bounded automation, and an unchanged initial kernel [1, 2]. The second report’s revised discussion explicitly incorporates corrections from the first. Their overlapping conclusions therefore should not be counted as independent experimental evidence that a language works.

The most useful conclusion is also their most demanding one: another vocabulary for refinement rows is not the missing contribution. Neither report supplies an implemented general property-aware authoring interface. Their compilation work establishes that small semantic combinators can be expressed in ordinary Lean. That is valuable, but does not establish predictable elaboration, statement fidelity, or lower author effort.

Juniper adopts the following corrections, not the stronger claims that earlier drafts sometimes made:

Sound finite certificates need not be complete. A sound checker may prove every claim it accepts while rejecting some true claims. A negative abstract witness requires a realizability argument and a transfer premise before it refutes a concrete specification. These are different obligations.

Equivalent designs are not identical encodings. Local propositions, subtypes, and records can represent related interfaces without being alpha-equivalent Lean terms. Proof irrelevance concerns proofs of the same proposition, not distinct structures or arbitrary changes of carrier.

A useful rule supplies sufficient premises. The premises of one interchange theorem, one inverse theorem, or one least-Lipschitz-constant argument are not automatically mathematically necessary conditions. An obligation diagnostic should name its selected route.

A proof checked in an inconsistent context does not test an arithmetic interface. The Frey helper should be exercised under satisfiable arithmetic assumptions, not only under a full package eventually shown impossible.

A statement can lose a premise during exposition. The reports identify an omitted finite/free assumption in a reconstruction of a representing algebra. This is evidence for preserving exact signatures in the default mathematical view, not evidence that the original Lean theorem was wrong [1, 2].

2.2 Representative code, not a caricature of Lean

The selected files expose several different costs. They also contain short proofs that Juniper should not make longer.

Table 1: Source workloads examined. File names are abbreviated here; full paths and revisions are recorded in the source manifest.

Source	Mathematical work	Authoring issue to target
ProveIt, autonomous derivatives	Induction, neighborhood equality, chain rule	Preserve a set-wide induction hypothesis and infer the locality bridge.
ProveIt, inverse derivative recursion	Formal Bell coefficients and analytic differential operators	Do not promote a formal coefficient result into an analytic derivative theorem without a bridge.
FLT, Frey package and integer model map	Bundled assumptions, divisibility, casts, coefficient equality	Infer guards of exact quotient transport, independently for each coefficient.
Leant, verification and post-verification	Callback acceptance, exact candidate receipts, scoped reordering	Preserve identity and distinguish receipts from proof objects.
Leant, Length contracts	Explicit provider laws and source-ordered relational contracts	Attach denotation theorems to abstract behavior rather than interpreting names as semantics.

The autonomous theorem in ProveIt already factors out a reusable mathematical argument. Its successor case obtains eventual equality from an identity throughout an open set and applies the chain rule. The wrapper for globally differentiable auxiliary functions is short [4]. The final FLT solution wrapper is also short; the larger file includes several reusable arithmetic and discriminant lemmas, not one enormous proof of the displayed final statement [7]. Whole-file line counts would be a misleading compression baseline.

The inverse-derivative file makes an unusually important distinction in its own documentation.

Its Bell-polynomial recursion is established in a formal power-series setting; the analytic operator theorem is proved separately. The file explicitly does not claim an analytic Bell-form recursion or analytic third and fourth derivative formulas from those formal coefficient results alone [5]. Juniper must preserve that separation even when the formulas look identical on the page.

2.3 Existing Lean features are baselines, not Juniper inventions

Lean’s subtypes already bundle a value and a proof, and their compiled representation does not add a runtime wrapper. An argument for new kernel refinements cannot therefore rest on a blanket claim that ordinary subtypes necessarily cost runtime allocation [11, 12]. The real questions are elaboration effort, transports, proof-term size, and interface stability.

Mathlib’s `fun_prop` already performs compositional property reasoning, uses registered theorems, and can discharge local side conditions. Its documentation also describes limits on transition-theorem chains because those chains enlarge search [15]. Juniper must compare its property engine with this functionality, not with a baseline stripped of it. Lean’s `grind` is another existing proof-producing service [16]. The `mvcgen` infrastructure already presents monadic programs through predicate transformers and verification conditions; round-three tests described it as experimental at their Lean 4.32.0 pin [17, 1, 2].

For historical orientation, bidirectional typing supplies the distinction between synthesizing information and checking against a demand; refinement-directed synthesis supplies a precedent for propagating specifications into program construction; Dijkstra monads supply a precedent for computing proof obligations from desired behavior [18, 19, 20]. Juniper’s proposed contribution is their disciplined integration with mathematical locality, ordinary Lean evidence, and explicit alternative frontiers, not the invention of these ideas.

3 What a mathematician would write

All listings in this section are **proposed mathematical surface syntax**. They are not accepted by the companion parser. Section 16 separately documents the implemented ground language.

3.1 State the mathematical move, retain its scope

A compact rendering of the autonomous-derivative argument could be:

```

theorem autonomous_derivatives:
  given U : Set Real with Open U
  given W, phi : Real -> Real
  given G, Gprime : Nat -> Real -> Real
  assume derivative W at x is phi(W(x)) on U
  assume G(0, W(x)) = W(x) on U
  assume for every n:
    derivative G(n) at W(x) is Gprime(n, W(x)) on U
  assume for every n:
    G(n+1, W(x)) = Gprime(n, W(x))*phi(W(x)) on U
  prove for every n: D^n W = G(n) o W on U
  by induction n keeping the domain U:
    zero: use the initial identity
    step:
      differentiate the preceding identity on U
      using the chain rule
      use the recurrence

```

The first derivative assumption is an assertion of a derivative, translated to `HasDerivAt`, not merely an equation about Lean’s total `deriv` operation. “On U ” expands to a quantified predicate. “Keeping the domain” means that the induction hypothesis quantifies over every point of U before the successor point is introduced. These are semantic choices that the compact view must preserve.

The phrase “differentiate the preceding identity” is a registered recipe. At a point $x \in U$ it requires eventual equality in $\mathcal{N}(x)$. A route from equality on U requires $U \in \mathcal{N}(x)$; openness and membership are one sufficient way to obtain that premise. The author need not spell out the filter lemmas, but the elaborated proof must use them or another explicitly permitted route.

3.2 Contracts at call sites

An operation can expose a property-implicit argument:

```

use cast_exact_integer_quotient(n, d)
  requires d divides n
  requires d != 0
  concludes cast(n / [Int] d) = cast(n) / [Rat] cast(d)

```

The operation is not a global reinterpretation of division. Integer division and rational division remain distinct. The two guards are sought in the local evidence index; a missing divisibility fact becomes an obligation. The library author writes the detailed theorem once. The user sees its mathematical interface at every use.

3.3 An incomplete step should remain visibly incomplete

Suppose $f = g$ on U and $x \in U$, but no neighborhood fact is available. A diagnostic could read:


```

Step: transport the derivative from f to g at x
Status: conditional; no theorem has been closed

Sufficient route 1: prove U is a neighborhood of x
Sufficient route 2: prove f and g agree near x directly

Open U would discharge route 1 using the known x in U.
No additional hypothesis has been added to the theorem.

```

An explanation can use a stronger recognizable premise such as openness, while retaining the exact required premise $U \in \mathcal{N}(x)$. A theorem may be true by a different route even when both displayed routes fail. “No route in this profile” is not “false.”

3.4 Construction is a separate act

A command asking for a sorting function has two outputs to obtain:

```

construct sort : List A -> List A
  satisfying for every xs:
    Permutation (sort xs) xs
    and Sorted order (sort xs)

```

First there must be a candidate function. Then there must be evidence of the exact universal specification for that function, under the fixed order. Tests may guide the search; a length law alone does not meet the specification. The source chooses whether executable code is required or a noncomputable existence witness is acceptable.

4 A property-aware typing discipline

4.1 Three environments, not one bag of hints

Let Σ be the selected mathematical signature: carrier types, universe parameters, operations, structures, and approved representation maps. Let Γ be Lean’s local context. Let $\mathcal{E}$ be an index of already checked evidence available in that context. A local view is written

$$\Sigma; \Gamma; \mathcal{E} \vdash t : A \triangleright \{P_1(t), \dots, P_k(t)\}.$$

Every row entry has an actual proof in Γ . The index may accelerate finding that proof, but is not a second source of logical authority.

An anchor is a typed term in this context, including the instantiated data arguments that determine its meaning. Printed variable names are not anchors. Neither is a hash alone. For example, changing the topology changes the proposition `Continuous f`; changing a scalar action changes the proposition that a map is linear. Those changes cannot be hidden behind proof irrelevance.

The local introduction and consequence rules are intentionally ordinary:

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash p : P(t)}{\Gamma \vdash t : A \triangleright \{P\}}, \quad \frac{\Gamma \vdash p : P(t) \quad \Gamma \vdash h : P(t) \rightarrow Q(t)}{\Gamma \vdash t : A \triangleright \{Q\}}.$$

Conjunction combines proofs about the same anchor. Equality elimination transports a predicate along an actual equality. A weaker observational relation requires a theorem naming the property that the relation preserves. A pointwise equality, an almost-everywhere equality, and agreement of finitely many coefficients are not interchangeable labels.

4.2 Local rows and exported types

The local notation does not force rebinding t to a subtype after every discovered fact. At a boundary, a package may be generated:

$$\{x : A \mid P_1(x) \wedge \cdots \wedge P_k(x)\}.$$

A mixed interface containing witness data uses a dependent record instead. For instance, a chosen basis is data; finite-dimensionality is a proposition. A chosen inverse algorithm is data; the statement that an element is a unit is a proposition. Adding a proof-only row must not silently choose either witness.

This is a genuine extension of the *surface typing discipline*: a call can be accepted by insertion of an inferred proof argument or a theorem-backed view adapter that plain use-site elaboration would not automatically discover. It is not an increase in what propositions Lean can express. The translation exposes every inserted argument and adapter as ordinary Lean terms.

4.3 Property-implicit arguments

A library operation can declare selected proof arguments as `requires` parameters. Its implementation is an ordinary dependent function with explicit proof binders. Juniper’s call elaborator tries to fill those binders from available evidence and registered theorem recipes. It must not turn every discovered fact into a global type-class instance.

The proof search may create fresh local proof metavariables, but must not solve a meaning-bearing variable by changing a carrier, a branch, a quantified object, or a measure. Lean’s synthetic opaque holes already distinguish certain deferred proof goals from ordinary inference holes; Juniper needs an additional audited staging policy, not a claim that all holes are currently indistinguishable [13].

The permissions should be explicit:

Hole class	What may be chosen	What remains fixed
Meaning inference	Ordinary elaboration resolves notation, structures, and indices before sealing.	The agreed mathematical statement after sealing.
Proof synthesis	A proof of an already fixed proposition.	Its proposition and the authorized context.
Construction	A value and evidence satisfying an explicitly delegated specification.	The specification, input domain, branch conditions, and execution requirement.

A constructive witness hole can affect later expressions that explicitly depend on its result. That does not allow it to instantiate a universally quantified parameter of the original theorem. The distinction is an information-flow rule for elaboration, not a test based on whether a variable’s printed name begins with a question mark.

4.4 Proof irrelevance and dependent transport

For the same proposition $P(t)$, different proof terms do not represent different mathematical objects. It does not follow that two structures on A are identical, or that $\{x : A \mid P(x)\}$ and $\{x : A \mid Q(x)\}$ are definitionally equal merely because P and Q are logically equivalent.

A particularly revealing case is an indexed object $v : \text{Vector}(A, n)$. If a proof establishes $n = m$, then a view of v at index m requires a dependent transport or a registered adapter; the fact that n and m happen to print similarly is irrelevant. Juniper should keep that transport in the evidence graph and explain it on demand, rather than requiring a new kernel conversion rule for every index identity.

5 Residual telescopes: the meaning of an unfinished proof

5.1 From a missing premise to a conditional derivation

An unfinished operation has the judgment

$$\Sigma; \Gamma; \mathcal{E} \vdash \text{step} \Rightarrow (P, \Delta, k), \quad \Gamma \vdash k : \Pi\Delta, P.$$

Here P is the locked target, Δ is a well-formed telescope of remaining obligations, and k is a derivation that works when those obligations are supplied. The target may depend on the telescope's earlier binders where that dependency is explicitly part of the construction request. A completion supplies a well-typed substitution θ for every binder of Δ ; the resulting proof is $k\theta$.

For a proof-only residual,

$$\Delta = (h_1 : P_1)(h_2 : P_2) \cdots (h_r : P_r),$$

this is simply a curried implication. The crucial rule is that k is not inserted into the fact index as a proof of P until its arguments exist. Juniper may store k as a conditional plan, with a different type and a different user-visible status.

5.2 Why a telescope is necessary

Not every residual is a set of independent propositions. Constructing a quotient factorization can leave

$$(g : Y \rightarrow Z), \quad (h : g \circ q = f), \quad (c : \text{Continuous}(g)).$$

The last two items depend on the first. Flattening them into unrelated existential claims would permit the equation to concern one function and continuity another. Similarly, a root value, a branch-membership proof, and a residual-zero proof must refer to the same root.

Juniper therefore uses two levels. The general interface is a dependent telescope. The finite antichain engine of Section 6 applies only after all data arguments relevant to its atoms are fixed. It neither invents witnesses nor pretends to solve dependent existential synthesis by propositional saturation.

5.3 Residual composition

Suppose a first recipe gives $k : \Pi\Delta, Q$ and a second, after receiving $q : Q$, gives $\ell : \Pi\Theta(q), P$. The composed residual retains the dependency:

$$\Pi\delta : \Delta, \Pi\eta : \Theta(k\delta), P.$$

Its proof is $\lambda\delta\eta. \ell(k\delta)\eta$. One cannot discharge an item of Δ by appealing to P unless an independent, well-founded proof supplies that item. This is the formal reason a result must not manufacture its own guard.

Proposition 5.1 (Residual discharge). *If $\Gamma \vdash k : \Pi\Delta, P$ and $\Gamma \vdash \theta : \Delta$ is a valid dependent substitution, then $\Gamma \vdash k\theta : P$. Branch-local variables and proof assumptions cannot escape if the substitution is checked in Γ .*

Proof. Apply the dependent function successively to the terms of θ , in telescope order. Each application has the type obtained by substituting the preceding arguments. The final type is P with the same authorized substitution. No free variable outside Γ can appear in a well-typed argument supplied by θ . $\square$

This elementary proposition does not verify an implementation of elaboration. Its purpose is to make the required implementation invariant explicit: an open residual is not a theorem, and completion is checked substitution rather than a change of status in a database.

5.4 Drafts, assertions, and publication

An author may keep a draft with open residuals. Publication as a verified theorem requires closure of the theorem and every displayed assertion presented as established, even an assertion unused in the final proof. Otherwise an attractive narrative could contain false intermediate claims beside a correct terminal theorem.

The renderer should distinguish an assumption made in the theorem statement, a locally assumed case, a proved assertion, a conditional plan, and an informal comment. Collapsing these into the same typography would defeat the entire design. Conditions changing the claim belong in the default view, not solely in a hover panel.

6 A bounded algorithm for alternative obligations

6.1 The finite problem

Trellis already gives a finite qualifier-closure theorem, and the two syntheses ask for a bounded rule generator and an effective obligation frontier [3, 1, 2]. Juniper extends that specific finite problem from *available facts* to *alternative sufficient additions*. The set-theoretic machinery is not claimed as a new foundation of logic; its role here is to make the proposed authoring contract exact and testable.

Fix a finite set A of grounded atoms, a set $K \subseteq A$ of known facts, a set $H \subseteq A$ of designated additional premises, and a finite collection R of rules

$$r : a_1 \wedge \cdots \wedge a_k \longrightarrow b.$$

A ground atom can contain an arbitrary local variable, provided its data arguments and context are fixed. Thus $x \neq 0$ with a local $x : \mathbb{R}$ is ground for this purpose; no enumeration of real numbers is involved. Each actual mathematical rule must be an instantiated Lean theorem. A rule with no premises is an available theorem.

Let $\text{Cl}_R(X)$ be ordinary forward Horn closure from X . Define the frontier of b by

$$\mathcal{F}(b) = \text{Min}_{\subseteq} \{S \subseteq H \mid b \in \text{Cl}_R(K \cup S)\}. \quad (1)$$

Only inclusion-minimal sets are retained. The empty set means no additional premise is needed. An empty *family* means no permitted set of additions suffices in this rule system. These cases must never share the same representation in an implementation.

H is a diagnostic vocabulary, not a new assumption context. A member may be a proposition the author could prove, an explicit premise the author is considering adding, or a currently open named obligation. The default interface should not offer the goal itself as a helpful repair. The mathematical definition permits that case when explicitly requested: its result is a conditional tautological route, not a proof.

6.2 Antichain propagation

For each $a \in A$, maintain an antichain $F(a)$ and a derivation node for every stored support. Initialize

$$F(a) \supseteq \{\emptyset\} \ (a \in K), \quad F(h) \supseteq \{\{h\}\} \ (h \in H),$$

then remove dominated sets. For every rule $a_1, \dots, a_k \rightarrow b$ and every tuple $S_i \in F(a_i)$, form $S = \bigcup_i S_i$. Insert S into $F(b)$ unless an existing support is contained in S ; on insertion remove strict supersets of S . Repeat until there is no change.

A newly inserted derivation node contains the instantiated rule and references to *already existing* premise nodes. It never points to a placeholder for its own conclusion. A removed antichain entry's node may remain in the derivation store if later nodes reference it. This preserves valid proof sharing while the presentation keeps only minimal supports.

```
frontier[known_atom] += { empty_support }
frontier[offered_atom] += { singleton(offered_atom) }
repeat:
  changed := false
  for each ground rule premises -> conclusion:
    for each tuple of currently available premise supports:
      support := union(tuple)
      if no stored support is a subset of support:
        retain a node referring only to older premise nodes
        remove strict supersets; insert support
      changed := true
until not changed
```

6.3 Soundness and termination

Theorem 6.1 (Sound conditional derivations). *Every stored entry $S \in F(b)$ admits a finite derivation of b from $K \cup S$ using R . Under an interpretation of the atoms as Lean propositions and of the rules as checked theorem instances, its derivation yields a proof of b under the corresponding explicit premises.*

Proof. Induct on node creation order. A known node uses its proof from K and has empty additional support. An offered node uses its corresponding premise and has singleton support. For a rule node, the induction hypotheses provide proofs of each premise under its recorded support. Under their union, all those proofs are available by weakening. Apply the instantiated rule theorem. All parent nodes precede the new node, so this argument is well-founded even when the rule graph contains cycles. Deleting a dominated display entry does not invalidate its retained node or any later use of that node. $\square$

Antichains themselves are not monotone under ordinary set inclusion: a smaller support can replace a larger one. The right termination measure uses upward closures. For a family $F \subseteq \mathcal{P}(H)$, put

$$\uparrow F = \{S \subseteq H \mid \exists T \in F, T \subseteq S\}.$$

Theorem 6.2 (Finite convergence). *Every accepted insertion strictly enlarges one upward closure $\uparrow F(a)$. Consequently there are at most $|A|2^{|H|}$ accepted insertions, including initialization, and a full-sweep implementation terminates.*

Proof. An insertion of S is rejected whenever some existing $T \subseteq S$ exists. If accepted, S was not in the old upward closure and is in the new one. Removing strict supersets loses no element of the upward closure because S covers all of them. Thus the upward closures increase monotonically, and each accepted insertion adds at least one pair (a, S) from a set of size $|A|2^{|H|}$. Between insertions a sweep visits finitely many rules and finite products of support families. A sweep without an insertion halts the algorithm. $\square$

The $|A|$ bound for ordinary fact closure does *not* carry over to support propagation. The extra exponential factor is real and should be part of the interface contract.

6.4 Relative completeness and schedule independence

Theorem 6.3 (Exact minimal frontier). *At a fixed point, $F(b) = \mathcal{F}(b)$ for every $b \in A$. Any fair rule schedule that eventually processes enabled support combinations obtains the same antichains, though its derivation graphs may differ.*

Proof. Soundness shows that every stored support is sufficient. For the other direction, suppose $b \in \text{Cl}_R(K \cup S)$. Induct on a finite Horn derivation of this fact. A known leaf is covered by the empty support. An offered leaf in S is covered by its singleton or by a smaller stored support. At a rule application, each premise has a stored support $T_i \subseteq S$ by induction. At a fixed point the union $\bigcup_i T_i$ is covered by some support $T \in F(b)$, so $T \subseteq S$.

Therefore every sufficient S contains a stored sufficient support. If S is inclusion-minimal, that support equals S . Conversely, if a stored support were not minimal, a smaller sufficient set would contain another stored support and contradict the antichain condition. This proves equality. Fairness ensures that a terminal state has the same closure property; the uniquely specified family in (1) then gives schedule independence. $\square$

The theorem is deliberately relative. It says nothing about omitted theorem instances, unoffered premises, a stronger semantic equivalence between offered propositions, or mathematical arguments outside R . “Minimal” does not mean a globally weakest hypothesis, the smallest number of words, or the fastest proof.

6.5 A checkable completeness certificate

A frontier producer should not be trusted merely because it returns a flag saying “complete.” There is a useful finite certificate characterization.

Theorem 6.4 (Frontier certification). *An antichain family F is the exact frontier if the following conditions hold:*

1. *every displayed support has a valid finite derivation from its support and K ;*
2. *every $a \in K$ is covered by the empty support, and every $h \in H$ is covered by $\{h\}$;*
3. *for every rule $a_1, \dots, a_k \rightarrow b$ and tuple $S_i \in F(a_i)$, some $T \in F(b)$ satisfies $T \subseteq \bigcup_i S_i$.*

Proof. Condition 1 implies soundness of every upward-closed family $\uparrow F(a)$. Conditions 2 and 3 make those families a rule-closed over-approximation of all seed supports. Induction on any finite Horn derivation therefore shows that every sufficient support is covered. Soundness and coverage give equality with the sufficient-support families; taking their minimal members gives (1). $\square$

The companion’s independent validator checks these conditions when a result is marked complete. For a budget-interrupted result it checks only the derivations and the current antichains. Such a result may contain useful sufficient routes, but does not claim global minimality or coverage even within the complete profile. This distinction is testable by deleting one alternative route or falsely setting a truncated result’s completion flag.

This certificate theorem is a paper theorem about a finite representation. The Python validator is not a formally verified checker, and the current Lean exporter does not certify frontier completeness in Lean. A small Lean implementation of this checker is a concrete next deliverable, more specific than another generic statement that a future elaborator should be sound.

6.6 Example and cost

Take known facts `OnSet`, `Member`, and offered premises `Open`, `DirectGerm`, `Positive`. Rules derive a germ from either the open-set route or direct evidence, derivative equality from the germ, nonzeroness from positivity, and a quotient step from both. The frontier is

$$\{\{\text{Open}, \text{Positive}\}, \{\text{DirectGerm}, \text{Positive}\}\}.$$

The prototype produces precisely these two supports. They do not say that openness or positivity is necessary in analysis. They say that these are the minimal sufficient *offered* combinations for these registered routes.

The exponential case is not pathological bookkeeping. Give each of k independent intermediate facts two alternative offered premises and make the goal require all k facts. There are 2^k pairwise incomparable sufficient sets. No algorithm explicitly listing every route can avoid output proportional to that number. An interactive system should normally display a small ranked selection and expose whether enumeration is complete. A bounded top- k display and an exact all-routes certificate are different products.

Finally, support size does not determine proof cost. Two routes with the same support may have very different terms or runtimes. Version one retains one derivation per support and makes no optimality claim. Later versions may retain a bounded Pareto set for support, proof size, and recipe policy, but their guarantees need separate statements.

7 Grounding rules without unbounded search

7.1 A finite automatic fragment must say how it becomes finite

The syntactic Horn calculus does not itself solve the difficult task of selecting useful Lean theorem instances. Juniper proposes a staged, bounded generator, not a universal principal-refinement algorithm.

At the beginning of a step, the elaborator records a finite term pool T_0 consisting of the target's typed subterms, relevant local declarations, and explicitly selected recipe parameters. Structures and universe arguments that determine propositions are resolved at this stage. A registry entry declares a conclusion pattern, its proof premises, data-input positions, and a bounded generation policy.

A conservative initial policy allows only data substitutions from a finite typed pool T . Recipe-specific constructors may extend the pool to depth d and total size B , with both limits explicit. A theorem that would require a new function, basis, or inverse witness creates a construction residual; it does not cause saturation to enumerate arbitrary Lean terms. Repeated generation of $f(x), f(f(x)), \dots$ therefore cannot continue without bound.

For a registry of m entries with at most k enumerable data positions, there are at most $m|T|^k$ raw assignments before typing and matching filters, when the pool is fixed. This is only a combinatorial bound on that specified enumeration. It excludes higher-order matching, reduction, instance resolution, checking proof arguments, and dependent well-formedness. Each successful assignment must yield a fully typed rule instance whose conclusion and premises are added to a finite manifest.

7.2 Two completeness flags

The front-end needs at least two separate indicators: *grounding coverage within the selected policy* and *frontier closure for the emitted manifest*. The frontier can be exact for the actual manifest

even when the generator stopped before enumerating every permitted assignment. Reporting only the second flag would make a partial mathematical search look complete.

The first implementation should favor explicitly registered, first-order-shaped theorem families and recipe-defined instantiations. Higher-order goals can be sent to ordinary Lean tactics or a synthesis service as a separate bounded action. A returned proof may seed a new snapshot; failure means only that the action did not return a proof.

The registry is not allowed to assert that a theorem instance exists merely because its name resembles a familiar lemma. Registering an entry checks its telescope, argument roles, target predicate, and selected structures. For external providers, the binding to the exact provider term is part of that check.

7.3 Demand-directed closure

After grounding, construct the backward dependency graph from the requested atom, retaining all premises of each retained rule. Saturating this closed subgraph preserves the exact frontier for the retained manifest. Heuristically dropping a rule may be a useful search optimization, but changes that manifest and must be reported as such.

Unknown facts should not trigger an indiscriminate search through the entire environment. A method can first ask for a narrowly targeted property: nonzeroness at a specified point, membership in a chosen domain, eventual equality at a specified filter, or the exact division law for a particular numerator and denominator. This is the operational meaning of property-directed authoring.

7.4 Scope and rewriting

Version one rebuilds the index at branch boundaries and after relevant edits. This costs more than an ideal persistent engine but gives a tractable correctness boundary. Parent facts can be reused in a child only through a valid context embedding. Reusing a fact in a sibling or after a deleted assumption is not permitted because the variable names still match.

A simplification from t to u has three possibilities: definitional conversion, an actual equality proof with transport, or a registered relation appropriate to the consumer. The old property is not silently rebound to u . A proof of $f = g$ almost everywhere can serve a compatible integral consumer; it cannot authorize arbitrary pointwise evaluation.

An implementation may hash the signature and context to route lookups and reject stale data quickly. The hash is not a mathematical proof of context equivalence. General reuse after an edit ultimately requires rechecking the term in the new context or checking a typed substitution. The prototype demonstrates only exact-profile rejection, not full Lean context transport.

8 Behavioral types beyond unary predicates

8.1 Total functions and restricted-domain functions differ

For a fixed total function $f : A \rightarrow B$, define

$$\text{Beh}(f; P, Q) := \forall x : A, P(x) \rightarrow Q(x, f(x)).$$

This is evidence about an already chosen total function. It is not generally the same object as a function defined only on a refined domain:

$$g : \Pi x : \{a : A \mid P(a)\}, \{b : B \mid Q(x.\text{val}, b)\}.$$

In particular, extending g to all of A may need extra data, decidability, or a default value. Juniper should use different surface constructs for a total function with a guarded law and for a function whose actual input is a subtype. It should not promise a total implementation by erasing a domain restriction.

8.2 Variance with a proof, not a slogan

Proposition 8.1 (Behavioral weakening). *Suppose $\text{Beh}(f; P, Q)$, $P'(x) \rightarrow P(x)$ for every x , and*

$$P'(x) \rightarrow Q(x, y) \rightarrow Q'(x, y)$$

for every x, y . Then $\text{Beh}(f; P', Q')$.

Proof. Given $P'(x)$, obtain $P(x)$ and hence $Q(x, f(x))$. Apply the postcondition implication at the same x and the actual output $f(x)$. Abstract over x and its premise. $\square$

The direction expresses contravariance of admissible inputs and covariance of promised outputs. The proof does not replace the implementation f . Juniper’s subtyping-like behavior is insertion of this proof, not a new equality between function types.

Proposition 8.2 (Composition with the actual intermediate value). *Suppose $\text{Beh}(f; P, Q)$, $\text{Beh}(g; S, T)$, and*

$$\forall x, y, P(x) \rightarrow Q(x, y) \rightarrow S(y).$$

Then $g \circ f$ satisfies

$$\forall x, P(x) \rightarrow Q(x, f(x)) \wedge T(f(x), g(f(x))).$$

Proof. Use the first contract to obtain $Q(x, f(x))$. The bridge supplies $S(f(x))$, so the second contract gives $T(f(x), g(f(x)))$. Pair the conclusions. No existentially chosen surrogate intermediate value is used. $\square$

8.3 Whole-function and relational contracts

Monotonicity compares two inputs; additivity relates three evaluations; continuity quantifies over neighborhoods; naturality relates a family of maps; exactness concerns a diagram. Juniper’s behavioral annotations must therefore admit arbitrary ordinary Lean predicates on these objects, not just a fixed menu of scalar input/output relations.

For example, a pair-valued partition operation can carry

$$\text{Permutation}(\text{left}(x) ++ \text{right}(x), x),$$

and joint membership conditions. Separate length bounds for the two outputs do not establish their combined content. Leant already has an explicit binary-product Length-contract vocabulary; Juniper should build on the source-ordered joint interface rather than reverting to independent output annotations [10].

For a representation map $v : A \rightarrow B$, *preservation* of a relation and *reflection* of it are different contracts:

$$R_A(x, y) \rightarrow R_B(vx, vy), \quad R_B(vx, vy) \rightarrow R_A(x, y).$$

A ring homomorphism preserves polynomial equality. Even an injective homomorphism need not reflect ideal membership after a change of coefficient domain. For example, X belongs to the ideal generated by $2X$ in $\mathbb{Q}[X]$, using multiplier $1/2$, but not in $\mathbb{Z}[X]$, since comparison of the coefficient of X would require an integer c with $2c = 1$. The operation’s type must name which relation it transports.

8.4 Behavioral abstractions and negative evidence

Suppose an abstraction $\alpha : X \rightarrow D$ and a provider model $F : D \rightarrow D'$ satisfy

$$\beta(f(x)) = F(\alpha(x)).$$

A proof of an abstract specification may transfer positively through this equation. A counterexample $d \in D$ is different: one must obtain an admissible x with $\alpha(x) = d$, and show that the abstract failure implies the concrete specification fails on that x .

The length abstraction makes the distinction vivid.

Proposition 8.3 (The image of list length). *For a type A , a natural number n is the length of a list over A exactly when $n = 0$ or A is inhabited.*

Proof. The empty list realizes zero. If $a : A$ exists, the list repeating a exactly n times realizes any n . Conversely, a nonempty list has a head, which witnesses that A is inhabited. A list of positive length is nonempty. $\square$

Thus an empty-output function on `List Empty` preserves every realizable input length, even though the abstract formula $0 = n$ fails at positive natural numbers. Positive length is not a concrete counterexample in that domain.

For affine length models $an + b$ and $cn + d$ with $a, b, c, d \in \mathbb{N}$, equality on all realizable lengths reduces to $b = d$ when A is empty, and to $a = c$ and $b = d$ when A is inhabited. In the inhabited case evaluate at zero and one, then cancel the equal constants; the converse is substitution. This is a complete criterion for that abstraction and domain, not a free theorem for arbitrary polymorphic Lean programs.

A failed candidate refutes that candidate only. Pruning an entire partially specified program requires a proof that every permitted completion violates the demand, or an explicit label that the pruning is heuristic. Juniper should preserve this distinction at the synthesis boundary.

9 Case study: exact arithmetic in the Frey model

9.1 The source already contains a property-rich object

The FLT source defines a `FreyPackage` containing integers a, b, c , an exponent p , nonzeroness, primality and a lower bound, a Fermat equation, a gcd condition, and congruences. Its integral and rational Weierstrass models use visually parallel coefficient expressions [6]. The map theorem proves that their meanings agree by proving the necessary divisibility before transporting integer division [7].

This is not an example of Lean being unable to express mathematical properties. It is an example of evidence already present in a rich package being reconstructed for a downstream operation. Juniper’s potential improvement is at the operation boundary.

9.2 Extract a satisfiable arithmetic interface

Consider the helper context

$$a, b \in \mathbb{Z}, \quad p \in \mathbb{N}, \quad a \equiv 3 \pmod{4}, \quad 2 \mid b, \quad p \text{ odd}, \quad p \geq 4. \quad (2)$$

It is inhabited, for instance by $(a, b, p) = (3, 2, 5)$. It does not contain the Fermat equation, primality, or the gcd assumptions. Set

$$N_2 = b^p - 1 - a^p, \quad N_4 = -a^p b^p.$$

Proposition 9.1 (Operation-specific divisibility). *If $a \equiv 3 \pmod{4}$, $2 \mid b$, p is odd, and $p \geq 2$, then $4 \mid N_2$. Independently, if $2 \mid b$ and $p \geq 4$, then $16 \mid N_4$.*

Proof. Write $b = 2k$. For $p \geq 2$, b^p is divisible by 4. Since $a \equiv -1 \pmod{4}$ and p is odd, $a^p \equiv -1 \pmod{4}$. Therefore $N_2 \equiv 0 - 1 - (-1) = 0 \pmod{4}$.

For $p \geq 4$, the factor $b^p = 2^p k^p$ is divisible by 16. Its multiple $-a^p b^p$ is consequently divisible by 16, without any hypothesis on the residue of a or the parity of p . $\square$

The separate interfaces are stronger engineering choices than giving both operations the entire package. In particular, $(3, 2, 3)$ is admissible for the first quotient but not the second. This is a simple example of meaningful premise minimization that the library author can prove before any automatic frontier search.

9.3 Exact quotient evidence

Use a relation

$$\text{ExactQuot}(n, d, q) : \quad dq = n.$$

Existence of such a q is divisibility. Uniqueness in $\mathbb{Z}$ additionally requires $d \neq 0$. To conclude that its image in $\mathbb{Q}$ is n/d , apply the homomorphism to $dq = n$ and divide by the nonzero image of d . These are distinct steps: preservation of the balance equation by a ring homomorphism is weaker than availability of division in the target.

For an arbitrary target ring, the appropriate condition is invertibility of the image of d , not merely its nonzeroness. A nonzero element need not be a unit; $2 \in \mathbb{Z}$ is the elementary counterexample. A regular element licenses cancellation but still need not have an inverse. Juniper should not collapse the three predicates into one “safe denominator” flag.

Under (2), the two quotients at $(3, 2, 5)$ are -53 and -486 . These values are finite sanity checks; Proposition 9.1, not the examples, proves the general divisibility.

9.4 A compact authoring sketch and its frontier

```
within integers and the selected embedding Int -> Rat:
  derive q2 := exact quotient (b^p - 1 - a^p) by 4
  derive q4 := exact quotient (-a^p * b^p) by 16
  transport both coefficients to Rat by exact quotient laws
  conclude equality of the Weierstrass models by coefficients
```

The source can preserve the already defined integer quotients as anchors rather than creating unrelated witnesses q_2, q_4 . A recipe either proves their balance equations or explicitly constructs a new quotient and proves equality with the source definition.

A useful frontier records the separate paths: the first coefficient depends on a mod-4 power argument; the second depends on the exponent lower bound. It should not silently use a terminal contradiction about the full Frey package. A named arithmetic recipe can restrict the ground manifest to the intended arithmetic theorems. That is a reproducibility constraint on the

Table 2: Boundary fixtures. Nonzero residues exhibit failure of the corresponding divisibility claim.

(a, b, p)	Changed condition	$N_2 \bmod 4$	$N_4 \bmod 16$
$(3, 2, 5)$	Positive fixture	0	0
$(3, 2, 4)$	Exponent not odd	2	0
$(3, 3, 5)$	b not even	3	7
$(3, 2, 3)$	Bound for the second quotient removed	0	8
$(1, 2, 5)$	Wrong residue of a	2	0

elaboration recipe, not a theorem that the resulting proof “essentially” uses only one mathematical idea.

These checks are under a satisfiable helper context. There is no need to manufacture a full Frey package to validate an arithmetic rule.

10 Case study: analytic inverse derivatives with certified algebra

10.1 A full mathematical contract

Let $U, V \subseteq \mathbb{R}$ be open, $f : \mathbb{R} \rightarrow \mathbb{R}$ be smooth on U , and $g : \mathbb{R} \rightarrow \mathbb{R}$ be continuous on V . Assume

$$g(V) \subseteq U, \quad f(g(y)) = y \quad (y \in V), \quad f'(x) \neq 0 \quad (x \in U). \quad (3)$$

This matches the shape of the analytic operator result in the examined ProveIt file [5]. It fixes the inverse branch through the actual function g and its local right-inverse law. Neither a formal series inverse nor an unspecified inverse notation is substituted for g .

Lemma 10.1 (First inverse derivative). *Under (3), g is differentiable at every $y \in V$ and*

$$g'(y) = \frac{1}{f'(g(y))}.$$

Proof. Fix $y_0 \in V$ and write $x_0 = g(y_0)$. For y near y_0 within V , continuity gives $g(y) \rightarrow x_0$. If $y \neq y_0$, the right-inverse identity implies $g(y) \neq x_0$. Hence

$$\frac{g(y) - g(y_0)}{y - y_0} = \left(\frac{f(g(y)) - f(x_0)}{g(y) - x_0} \right)^{-1}.$$

The inner quotient tends to $f'(x_0)$ by differentiability of f , and that limit is nonzero. Continuity of inversion at a nonzero real therefore gives the claimed limit. Openness of V makes this an ordinary two-sided derivative at y_0 , rather than a derivative restricted to an unspecified subset. $\square$

The source’s autonomous theorem provides the reusable mechanism for higher derivatives. A family G_n with $G_0(x) = x$ and $G_{n+1} = G'_n/f'$ gives $g^{(n)} = G_n \circ g$. The proof differentiates an identity valid throughout the open set V , not just at the current point [4, 5].

10.2 A differential-polynomial recurrence

Let u_j denote a formal variable corresponding to $f^{(j)}$. On the polynomial ring in the variables $u_1, u_2, \dots$ over $\mathbb{Z}$, define the derivation

$$D(u_j) = u_{j+1}, \quad D(P) = \sum_{j \geq 1} u_{j+1} \frac{\partial P}{\partial u_j}.$$

Each polynomial involves only finitely many variables, so the displayed sum is finite. Define

$$P_1 = 1, \quad P_{n+1} = u_1 D(P_n) - (2n-1)u_2 P_n \quad (n \geq 1). \quad (4)$$

Theorem 10.2 (Inverse derivatives from differential polynomials). *Under (3), for every $n \geq 1$ and $y \in V$,*

$$g^{(n)}(y) = \frac{P_n(f'(g(y)), f''(g(y)), \dots, f^{(n)}(g(y)))}{f'(g(y))^{2n-1}}. \quad (5)$$

Proof. For a polynomial P , ordinary differentiation and the chain and product rules give

$$\frac{d}{dx} P(f'(x), f''(x), \dots) = (DP)(f'(x), f''(x), \dots).$$

One can prove this identity structurally for constants, variables, addition, and multiplication. This is the semantic theorem required of a differential-polynomial reifier.

Set

$$R_n(x) = \frac{P_n(f'(x), \dots, f^{(n)}(x))}{f'(x)^{2n-1}} \quad (x \in U).$$

Smoothness and nonzeroness of f' imply that these expressions are differentiable on U . The quotient rule and (4) yield

$$\begin{aligned} \frac{R'_n(x)}{f'(x)} &= \frac{f'(x)(DP_n)(f'(x), \dots) - (2n-1)f''(x)P_n(f'(x), \dots)}{f'(x)^{2n+1}} \\ &= R_{n+1}(x). \end{aligned}$$

The first-derivative lemma establishes $g' = R_1 \circ g$. If $g^{(n)} = R_n \circ g$ throughout V , differentiating at any point in V gives

$$g^{(n+1)} = (R'_n \circ g)g' = (R'_n/f') \circ g = R_{n+1} \circ g.$$

The identity being differentiated is local on an open set, and $g(V) \subseteq U$ licenses each occurrence of the guard. Induction proves (5). $\square$

This is a paper-level analytic derivation included here, not a claim that the examined repository already contains a kernel-checked theorem with this exact statement. It also does not follow merely from displaying the same formulas for formal Taylor coefficients. The analytic interpretation, the local inverse argument, and the quotient guards do the missing work.

10.3 The first formulas and checkable invariants

The recurrence gives

$$\begin{aligned} P_1 &= 1, & P_2 &= -u_2, \\ P_3 &= 3u_2^2 - u_1u_3, \\ P_4 &= -15u_2^3 + 10u_1u_2u_3 - u_1^2u_4. \end{aligned}$$

Substitution in (5) yields the familiar first four inverse derivatives with their analytic hypotheses intact. The companion computes through P_6 , using exact integer coefficients.

Proposition 10.3 (Structural invariants of the polynomial family). *For $n \geq 1$, every monomial of P_n has ordinary degree $n-1$ and weighted degree $2n-2$ when u_j has weight j . The polynomial involves no variable beyond u_n . The coefficient of u_2^{n-1} is*

$$(-1)^{n-1}(2n-3)!!,$$

with $(-1)!! = 1$. For $n \geq 2$, the coefficient of $u_1^{n-2}u_n$ is -1 .

Proof. The assertions about degree, weighted degree, and largest index hold for P_1 . The derivation D preserves ordinary degree, raises weighted degree by one, and raises the largest possible index by at most one. Multiplication by u_1 then raises ordinary degree by one and weighted degree by another one. Multiplication by u_2 in the second term of (4) has the same degree effects. Induction proves the three invariants.

The first term of (4) is divisible by u_1 and cannot contribute to a pure power of u_2 . Its coefficient is therefore multiplied by $-(2n - 1)$ at each step, starting from 1 at $n = 1$, giving the double factorial.

For $n \geq 2$, the degree invariants show that a monomial involving u_n can only be $u_1^{n-2}u_n$: using u_n consumes weight n , and the remaining $n - 2$ factors must each have the minimum weight one. In the recurrence the only contribution to $u_1^{n-1}u_{n+1}$ comes from differentiating this u_n factor and multiplying by u_1 . Its coefficient is inherited unchanged from P_n . The initial coefficient in $P_2 = -u_2$ is -1 . $\square$

These invariants are useful contracts for a symbolic backend. They detect some errors cheaply, but do not certify an entire polynomial identity by themselves: a wrong coefficient can preserve both degrees. The proof-producing polynomial comparison still has to check the actual expression.

10.4 How Juniper would factor the work

```
family inverse_derivative_formula(n) for n >= 1:
  use the fixed local inverse g of f from V into U
  retain f' nonzero on U
  define P by the differential-polynomial recurrence
  prove D^n g = (P(n)[derivatives f] / (f')^(2*n-1)) o g
    by induction using the inverse derivative rule

specialize n = 4
  normalize the polynomial certificate
```

The mathematical insight is the recurrence and the inverse branch. The analytic library proves the semantic derivative and locality rules. The symbolic backend expands or normalizes P_n . The property layer obtains $f'(g(y)) \neq 0$ from the already recorded map-into-domain and nonvanishing contracts whenever needed. The source author does not repeatedly reconstruct that implication.

A fixed-order polynomial calculation is a good CAS task. An arbitrary-order theorem is not proved by checking six examples; it uses the induction above or its formal counterpart. Juniper's interface should make that distinction explicit.

10.5 Negative neighbors

For $f(x) = x^3$, the inverse is continuous but has no finite derivative at zero; $f'(0) = 0$ blocks the inverse-derivative recipe. Nonzeroness is not cosmetic.

Continuity of the chosen branch also matters. On $V = (0, \infty)$ let $f(x) = x^2$ on $U = \mathbb{R} \setminus \{0\}$, and choose $g(y) = \sqrt{y}$ for rational y and $g(y) = -\sqrt{y}$ for irrational y . Then $g(V) \subseteq U$, $f \circ g = \text{id}$, and f' is nonzero on U , but g is discontinuous and the derivative conclusion fails. A branch-selection contract must not be inferred from the equation alone.

Finally, agreement at one point cannot license differentiating an identity: $f(x) = x$ and $h(x) = 0$ agree at zero but have different derivatives there. Juniper's locality index prevents this invalid shortcut before any polynomial normalization is attempted.

11 Case study: descent through a quotient

11.1 An operation with a universal property

A language should support mathematical constructions beyond arithmetic normalization. Quotient descent is a useful test because it combines a relational law, a construction, uniqueness, and an optional topological guarantee.

Let $q : X \rightarrow Y$ be surjective and let $f : X \rightarrow Z$. The required behavioral law is constancy on fibers:

$$\forall x, x', q(x) = q(x') \rightarrow f(x) = f(x').$$

Theorem 11.1 (Descent and its topological extension). *There is a unique function $\bar{f} : Y \rightarrow Z$ satisfying $\bar{f} \circ q = f$ exactly when f is constant on the fibers of q . If X, Y, Z are topological spaces, Y has the quotient topology for q , and f is continuous, then this factor $\bar{f}$ is continuous.*

Proof. A factorization implies fiber constancy by applying $\bar{f}$ to an equality of q -images. Conversely, for each y , choose a preimage x and define $\bar{f}(y) = f(x)$. Fiber constancy makes the value independent of the choice. The defining equation follows immediately, and surjectivity shows that any two factors agree on every y .

For continuity, the quotient-topology property says that a map $h : Y \rightarrow Z$ is continuous exactly when $h \circ q$ is continuous. Apply it to $h = \bar{f}$ and use the defining equation and continuity of f . $\square$

This proof states precisely where choice enters for a bare arbitrary surjection. When Y is represented as an actual quotient type with its eliminator, a respectfulness proof can instead feed quotient elimination directly. Those are different implementation interfaces. A proposition asserting surjectivity is not automatically an executable preimage-selection algorithm.

11.2 A property-directed construction

```
construct fbar by descending f along q
  requires f constant on the fibers of q
  returns fbar with fbar o q = f
  derives uniqueness using surjectivity of q
  derives continuity using the quotient topology and continuity of f
```

The primitive mathematical move is “descend.” The library recipe knows its premises and constructs the factor. The behavior of the result is then available to later consumers. For a selected actual quotient, the author need not repeatedly write representative choices and independence proofs.

The residual telescope preserves the same $\bar{f}$ across the factorization and continuity obligations. It must not use a factorization for one candidate and a continuity theorem for another. Nor may continuity of f be promoted to continuity of $\bar{f}$ without the quotient-topology property.

This example sharpens the boundary between local properties and object-changing operations. Learning that f is fiber-constant does not create a new f . Descending it produces a new function on a different carrier, together with a relation to the old one. Both acts can use one evidence infrastructure, but they are not the same typing rule.

11.3 Useful failure cases

If q identifies points at which f has different values, descent cannot satisfy the factorization. If q is not surjective, agreement after precomposition need not imply uniqueness of a factor on the rest of Y . If the topology on Y is changed, the continuity conclusion needs a new proof. Juniper should report the failed fiber law, missing coverage, or changed topology respectively, rather than replacing the quotient or selecting a convenient topology behind unchanged notation.

12 A hybrid proof assistant and computer algebra system

12.1 A computation should return a typed mathematical result

A CAS request should be a particular operation with a particular output contract: normalize a polynomial, produce a factorization with a reconstruction identity, isolate roots with interval and multiplicity evidence, compute a finite jet, or generate a candidate for a universal property. The output type must distinguish a witness from a complete classification, and an approximation from an exact equality.

An untrusted producer may return a value v and certificate c . A checked interface has the shape

$$\text{check}(s, v, c) = \text{true} \implies \text{Rel}(\llbracket s \rrbracket, v),$$

where s is typed syntax with a specified denotation and Rel is the promised relation. A source-to-syntax bridge must also establish that $\llbracket s \rrbracket$ is the original expression, under the original valuation and structures.

The correctness theorem of an algorithm does not establish that an external executable returned the value it claims. Either recompute the checker in the trusted proof path, retain a checkable certificate of the result, or explicitly adopt a larger execution-trust profile. A native success receipt alone is not a proof of the requested mathematical relation.

12.2 Separate algebraic normalization from analytic licensing

The inverse-derivative example has a useful two-part contract. A polynomial normalizer proves an identity in $\mathbb{Z}[u_1, \dots, u_n]$ or a selected commutative ring. An analytic theorem interprets the variables as actual derivatives and discharges nonzeroness and locality premises. The backend should not receive the authority to infer these premises from a successful algebraic computation.

The same discipline prevents familiar errors. Replacing $\sqrt{x^2}$ by x over the reals needs a sign condition; without it the natural result is $|x|$. Clearing denominators for a rational-function identity preserves an equality only under the appropriate guards or as an identity in a separately defined localization. A coefficient computation for a formal series is not automatically a convergent analytic expansion.

Use existing proof-producing Lean algebraic tactics where they already suffice. Juniper does not require a new polynomial checker simply to reproduce `ring`. A new backend is justified by a concrete workload, a smaller or faster certificate path, or an operation the existing baseline does not cover.

12.3 Locality, precision, and measure are contract parameters

A finite jet through order N is an equivalence modulo X^{N+1} , not an exact series. Formal differentiation guarantees agreement modulo X^N , losing one order. This follows because differentiating

a multiple of X^{N+1} leaves a multiple of X^N . Inversion or composition may impose unit and constant-term conditions. Their output precision is part of their behavioral type, not a display preference.

For integration, an almost-everywhere equality is useful precisely because the consumer respects it. One sufficient real-valued integration interface takes measurable functions, an almost-everywhere equality, and integrability of one function, and returns integrability of the other and equality of their integrals. It does not manufacture pointwise equality. The measure is a fixed parameter of every premise.

Pointwise convergence alone cannot authorize interchange of limit and integral. On $[0, 1]$, the functions $h_n(x) = n\mathbf{1}_{(0, 1/n)}(x)$ converge pointwise to zero while each integral is one. A dominated-convergence recipe must request domination and the relevant measurability and integrability conditions, or select another proved theorem with different sufficient premises. An obligation frontier displays such alternatives; it does not declare one theorem's premises to be the only possible justification.

12.4 Certificate provenance and result strength

The retained result should identify its source expression, coefficient domain, valuation, checker theorem, checked execution, and precise relation. A certificate for an expression with the same printed variables but a different valuation is not transferable without a bridge. A rational ideal-membership witness cannot be replayed as an integer witness. A candidate root cannot stand in for coverage of all roots.

These requirements are not objections to aggressive automation. They allow aggressive untrusted search precisely because acceptance is narrow and explicit. Search can be expensive, heuristic, parallel, or model-guided while its successful result is checked against a fixed mathematical contract.

13 Integrating Leant without weakening its boundaries

13.1 Use existing candidate identity infrastructure

At the inspected Leant snapshot, `Verification.hs` explicitly describes `Verified` as a receipt that a supplied callback accepted a particular candidate, not as a behavioral certificate or a kernel proof. `PostVerification.hs` scopes candidate occurrences by a fresh epoch and validates exact bounded permutations of their receipts [8, 9]. This is useful infrastructure: Juniper should preserve the candidate and origin identities rather than introduce an independent loosely keyed evidence store.

`Length/Contract.hs` is equally explicit about its role. Provider laws are passive assumptions supplied by an integration layer; handoff checks exact identities and provenance but does not infer semantics from provider names [10]. Juniper's proposed addition is a proof-linked registration interface that supplies an actual theorem about the exact provider term. It is not a criticism of the passive vocabulary for failing to be an autonomous theorem prover.

The reports discuss additional Leant work at another explicit revision. This article's direct code observations refer to the recorded snapshot, not an unqualified claim about every later commit [1, 2]. The integration requirements below remain relevant regardless of which component currently performs a particular check.

13.2 A proof-carrying synthesis request

A request should contain a locked target type, a behavioral proposition with a hole for the candidate, fixed operations and domains, a permitted component library, construction permissions, and a checking profile. Its result should separately retain

$$(t : A), \quad (p : B(t)), \quad \text{origin information and replay dependencies.}$$

A candidate can typecheck without satisfying B . A receipt can attest to a callback without retaining p . A proof of a related abstraction can be useful without yet being a proof of $B(t)$. These statuses should not collapse into a single Boolean called *verified*.

In a proof-search use case, the construction target may itself be a proposition. A tactic suggestion remains a proposal until it is replayed against the exact goal and produces an accepted proof. A successful search must not retroactively instantiate a meaning-bearing parameter to make that goal easier.

13.3 Requirements can guide candidate generation

Suppose a composition $g \circ f$ is sought with target contract B . A registered composition recipe generates demands on f , g , and their connecting property. The frontier can then distinguish a candidate lacking only a nonvanishing proof from one whose behavioral shape is incompatible with every registered route.

This resembles specification decomposition in refinement-directed synthesis [19], but the whole Lean proposition language is not thereby made decidable. For the predictable layer, the decomposition is a finite manifest of theorem instances. Outside it, synthesis may propose terms and proofs with no completeness claim.

Candidate ranking can use route count, unresolved construction holes, or an estimated proof cost. It must not report an unproved candidate as correct merely because its remaining frontier looks easy. Conversely, a missing route in one abstraction should not necessarily discard a candidate that another proof method could verify.

13.4 Positive, negative, and bounded evidence

The protocol should return distinct constructors for an exact-target proof, a concrete counterexample with a proof of failure, a bounded test outcome, an unsupported abstraction case, and resource exhaustion. An abstract counterexample becomes concrete only through a realization theorem or a checked actual input. A candidate rejected by one certificate does not imply that its mathematical specification is false.

A future integration can retain proofs rather than only transport them transiently through callbacks. Replay should use the stored theorem, exact instantiations, and denotation bridges, without rerunning the synthesis model. When a dependency changes, recheck or invalidate the result; do not repair its statement silently.

14 How far should Lean’s type system be extended?

14.1 Recommended first layer: an elaboration extension

The first Juniper implementation should add property-aware checking judgments, local evidence lookup, property-implicit parameters, explicit residuals, and theorem-backed coercion recipes. Its

output should use existing dependent functions, propositions, subtypes, structures, and equality elimination. Lean’s extensible elaboration architecture is designed to translate richer source constructs into core expressions; the source extension does not by itself require a new trusted term former [14].

This recommendation should not be confused with doing nothing to the type system. The author experiences a richer notion of admissible use: an operation expecting a nonzero real can accept an expression with a proved positive lower bound after constructing the necessary evidence. A function with a behavioral specification can meet a demand after checked weakening. These are meaningful typing features, even though their meaning is compilation to ordinary proof arguments.

The fixed semantic target is essential. Property inference must not make the authoring experience more permissive by weakening the proposition being proved. The intended improvement is that more evidence is reconstructed automatically for the same proposition.

14.2 An optional native refinement former

A research branch could introduce a native former $\text{Ref}(A, P)$ with a value, evidence, and a projection to A . Candidate rules would mirror subtype introduction and elimination. Same-carrier refinement subsumption would require an explicit proof of $\forall x, P(x) \rightarrow Q(x)$, not invoke arbitrary theorem search inside conversion.

The immediate question is what this buys over the existing encoding. Since subtypes already erase their proof fields at runtime, plausible benefits concern elaborator ergonomics, generated term size, conversion costs for dependent interfaces, or better internal sharing. These benefits must be demonstrated on actual workloads. A prettier surface notation alone is insufficient justification for modifying the trusted kernel.

A well-defined experimental comparison would use identical mathematical statements and generated proofs in both encodings, measure elaboration and checking cost separately, and identify the exact term pattern responsible for any difference. If the problem disappears through ordinary record design or a better adapter theorem, that is the preferred solution.

14.3 What should not enter definitional equality

Juniper should not initially make arbitrary refinement implication part of definitional equality. Proving $P \rightarrow Q$ in unrestricted Lean mathematics is an open-ended proof problem, not a predictable conversion procedure. Nor should equality proofs generally cause the kernel to identify arbitrary dependent types without explicit transport.

Similarly, a behavioral equivalence of two functions does not mean the functions are computationally interchangeable in every dependent context. A theorem that two maps have the same length effect does not identify their outputs; a theorem that two representations induce the same finite jet does not identify their infinite values.

None of this says richer type theories are intrinsically impossible or unsafe. It says they must be specified and evaluated as separate foundations work, rather than smuggled into a convenience feature. The normal authoring path should keep undecided proof search outside the conversion mechanism.

14.4 Proof obligations for a kernel experiment

A native refinement or behavioral primitive would need precise universe rules, substitution lemmas, subject reduction, interaction with proof irrelevance and elimination, and a conservative translation

or another explicit justification of consistency relative to the chosen foundation. Its implementation needs independent validation as well as a paper calculus. Lean4Lean provides an existing line of work on an external checker and formalized portions of Lean’s metatheory; it is relevant infrastructure, not a certificate automatically covering any proposed extension [21].

For now, Juniper’s finite frontier calculus requires no new kernel feature. Its explicit completeness checker is itself a candidate for formalization in ordinary Lean. That is a smaller and more immediately testable task than changing the type theory to perform unspecified mathematical entailment.

15 An implementation architecture with explicit acceptance criteria

15.1 Seven components and one trusted endpoint

A realistic implementation can be divided into the following modules:

Component	Responsibility
Semantic model	Store fixed targets, structures, contexts, anchors, and permitted construction holes.
Registry	Validate theorem recipes, argument roles, provider bindings, and transport contracts.
Grounding	Produce a finite typed manifest and a separate grounding-coverage status.
Frontier	Produce sufficient supports, derivation graphs, and optional closure certificates.
Replay	Construct Lean terms from the chosen recipe and check exact target types and dependency policy.
Surface elaborator	Translate mathematical commands, scope, induction, and construction into these requests.
Renderer	Show the same mathematical claims and their statuses, with expandable evidence.

The trusted logical endpoint is acceptance of ordinary Lean evidence under the declared axiom policy. The grounding engine, search scheduler, CAS producer, and language model can all be untrusted producers. The renderer is not logically trusted by the kernel, but it is part of the user-facing correctness boundary: a misleading display can be harmful even beside a valid proof.

15.2 Three forms of fidelity

Logical fidelity means that the accepted term proves the elaborated proposition. **Statement fidelity** means that the proposition and its displayed hypotheses are what the author approved. **Recipe fidelity** means that a requested method is actually the recipe replayed by the elaborator. These are separate acceptance conditions.

A source request such as “by the chain rule” should select a typed recipe whose slots are filled by subordinate proofs. It should not be implemented by accepting any proof and attaching a chain-rule label. A decorative occurrence of a theorem somewhere in a proof term does not establish that the argument depends essentially on that theorem. Juniper should make the narrower, checkable promise about replayed recipes.

Minimal supports are computed within a selected recipe profile. Otherwise a stronger unrelated theorem might eliminate the displayed premises and replace the author’s intended argument. Later products can compare alternative recipes explicitly; the profile selection must remain visible when it affects method fidelity.

15.3 Avoiding a theorem-backed but misleading narrative

The generated proof should be accompanied by a statement manifest containing its elaborated type, structure selections, introduced assumptions, theorem dependencies, and displayed intermediate claims. A source macro must account for the claim it displays, not just return a proof of a convenient terminal statement.

At publication, every displayed mathematical assertion marked as proved is checked. The final theorem cannot launder an unused incorrect assertion. At import changes, a statement diff is separate from a proof replay diff. A proof that still checks after the elaborator chose a different topology is not a successful unchanged replay.

These principles do not demand that every kernel detail be printed in a textbook proof. They demand that hypothesis changes, branch choices, restricted domains, and incomplete statuses be legible without inspecting an implementation trace.

15.4 A relative elaboration theorem

Theorem 15.1 (Soundness of the specified evidence recipes). *Assume that source meanings and data choices have been fixed, every initial evidence item is checked in its actual context, each recipe is an ordinary typed Lean construction, every external result is connected to its exact target by checked evidence, and every residual is discharged by a checked substitution. Then each completed Juniper proof assembled from these recipes translates to an ordinary Lean proof of its declared elaborated target. The recipe rules themselves require no new axiom.*

Proof. Induct on the finite recipe derivation. Leaf nodes use checked initial evidence. Property consequence and behavioral rules use dependent application of checked theorems. Conjunction, implication, and universal introduction use the corresponding ordinary constructors and binders. Equality or relational transport uses its checked bridge. External results use their checked denotation and certificate premises. Residual composition and discharge are justified by Section 5. At every node the output term is checked at the declared target in the authorized context. The axiom dependencies are those of the supplied evidence and environment. $\square$

This is a theorem about the specified architecture, not a verified compiler implementation. It does not prove that a parser captures human intention, that a statement manifest is displayed honestly, or that an implementation enforces every staging rule. Those require implementation tests and, where possible, formal verification of the relevant components.

16 The delivered executable slice and what was tested

16.1 An implemented language small enough to describe completely

The file `companion/juniper_frontier.py` implements the following ground language:

```

profile LocalDerivative
atoms OnSet Open Member Germ DerivEq Positive Nonzero QuotientReady
atoms DirectGerm NeverNeeded

given OnSet Member
offer Open DirectGerm Positive
rule neighborhood: OnSet Open Member -> Germ
rule direct: DirectGerm -> Germ
rule derivative: Germ -> DerivEq
rule nonzero: Positive -> Nonzero
rule quotient: DerivEq Nonzero -> QuotientReady
rule circular: NeverNeeded -> NeverNeeded
need QuotientReady
need NeverNeeded

```

Comments begin with `##`. Identifiers contain ASCII letters, digits, and underscores and begin with a letter. The parser rejects unknown directives, undeclared atoms, repeated declarations where disallowed, malformed rules, overlapping `given`/`offer` sets, and missing goals. Rules may have zero premises. A repeated premise is ordinary reusable propositional evidence, not a linear resource.

The atom names are opaque identifiers. `Germ` does not cause the program to read a Lean topology, and `Positive` does not invoke an inequality solver. The mathematical interpretation of those names is deliberately outside this prototype. Its implemented functionality is the finite algorithm of Section 6, not the larger syntax of Section 3.

16.2 Derivation validation and export

The solver retains node kind, conclusion atom, support, rule identifier, and parent identifiers. The validator checks support consistency and that every parent precedes its child. For complete results it additionally checks seed coverage and closure of the frontier under every rule combination, as in Theorem 6.4. A changed profile fingerprint causes rejection; this is a conservative exact-profile check, not a proof of semantic equivalence between edited Lean contexts.

The exporter emits a Lean theorem for each retained route, using propositions `A0`, `A1`, and so on, explicit rule hypotheses, given hypotheses, and exactly that route's offered hypotheses. Intermediate `have` declarations share the reachable derivation graph. There are no `sorry` or `axiom` commands in the generated file.

Crucially, the exporter proves generic implications *assuming the supplied rules*. It does not instantiate those rules with actual calculus theorems. This is an honest source boundary: the exporter tests the shape of proof assembly, while a mathematical reifier and Lean registry remain future implementation work.

No Lean or Lake executable was available in the working environment. `Generated.lean` was therefore **not compiled**. The Python derivation validator and exhaustive finite oracle are not substitutes for a Lean kernel check. The generated file is included for inspection and later compilation, with that status recorded in its header.

16.3 Executed results

All reported results below were produced by the included scripts; none is a human productivity measurement.

Table 3: Executed companion checks. Counts are deliberately separated rather than aggregated into a misleading single score.

Check family	Count	What the check establishes for the sampled data
Unit test methods	32	Expected behavior of parsing, closure states, mutation rejection, and export shape.
Generated ground profiles	400	Seven atoms, four offered premises, twelve rules per profile, deterministic random seed.
Exhaustive support assignments	6,400	Independent ordinary-closure oracle over every subset of the four offers.
Goal frontiers compared	2,800	Exact agreement between antichain output and oracle for all seven atoms.
Reversed rule schedules	400	Same support families with all rule orders reversed.
Admissible Frey triples	189	Exact integer divisibility checks in the stated finite ranges.
Inverse-polynomial formula comparisons	4	First four polynomials match separately written expected forms.
Inverse-polynomial invariant checks	38	Degree and weighted degree for the nineteen monomials through order six.

The unit suite includes seedless and seeded cycles, nullary rules, dominated and incomparable supports, absent offers, all displayed demands, resource exhaustion, changed profiles, removed assumptions, corrupted supports, self-reference, wrong rule applications, a falsely asserted completeness flag, and an omitted alternative route. All 32 methods passed. The randomized experiment uses seed 20260906; ordinary forward closure is implemented separately from antichain propagation.

The arithmetic probe ranges over $-13 \leq a \leq 13$, $-8 \leq b \leq 8$, and $4 \leq p \leq 9$, retaining exactly the triples satisfying (2). The inverse-polynomial program uses sparse exponent vectors and exact integer coefficients. Neither script is formally verified; the paper proofs explain the mathematical properties that a verified implementation should establish.

16.4 The exponential family is visible even in a tiny example

Table 4: Independent binary choices in an explicit frontier. Timing data are retained in the JSON log but are not used as a performance claim.

Choice pairs	Offered atoms	Minimal routes	Retained nodes	Rule-combination attempts
1	2	2	6	8
2	4	4	12	16
3	6	8	20	28
4	8	16	32	48
5	10	32	52	84
6	12	64	88	152
7	14	128	156	284
8	16	256	288	544

The experiment confirms the deliberately constructed output growth, not an asymptotic discovery. It also shows why a user interface should not eagerly print all routes. The reference uses a simple full-sweep algorithm; it is not an optimized production implementation.

16.5 Evidence ledger

The delivered artifacts implement and execute ground parsing, finite antichain propagation, derivation and closure validation, and generic Lean-source export. The article proves the finite algorithm’s mathematical guarantees, the residual-discharge rule, behavioral rules, and the worked mathematical propositions.

The following are *not* delivered as working components: Lean-expression reification, a typed theorem registry, bounded Lean theorem grounding, property-implicit elaboration, the mathematical surface parser, a proof-aware renderer, native verified CAS integration, or a compiled proof of the frontier algorithm. No repository-wide build, end-to-end ProveIt or FLT rewrite, or usability gain is claimed. This boundary is a development specification, not a reason to treat the entire design as implemented.

17 Evaluation and a falsifiable development program

17.1 Separate language gains from library gains

The controls proposed in the syntheses should be retained [1, 2]. Compare:

1. the original Lean development;
2. Lean with the same improved mathematical library;
3. Lean with that library and the same search, CAS, and synthesis services;
4. the property-aware authoring interface using those identical services;
5. a narrative surface over the same property-aware implementation.

Add a read-only mathematical renderer for the strongest ordinary-Lean baseline. Otherwise a more attractive display can be mistaken for a better authoring language, and a better library can be mistaken for a type-system improvement.

Primary measures should include author interventions, time actively spent resolving errors, successful statement reconstruction by readers, proof repair after controlled changes, and the frequency of silently misunderstood hypotheses. Source length is secondary. Also measure elaboration latency, kernel-checking time, proof size, retained evidence size, and failure locality. Do not aggregate these into one score without reporting the components.

17.2 Concrete acceptance milestones

First, mechanize the finite slice. Implement the antichain data and the certificate checker in Lean, prove Theorem 6.4 for the representation, compile the generic export, and verify rejection of corrupted certificates. This closes a precise gap in the current companion.

Second, connect one real operation. Add a typed registry and actual Lean grounding for the two exact quotient coefficients. Compare against a library-only wrapper. The positive run must use the satisfiable arithmetic helper, and the negative fixtures must fail at the correct guards.

Third, validate locality. Connect the autonomous-derivative recipe with an explicit domain-wide induction hypothesis. Mutating equality on a neighborhood to equality at one point must not close the derivative transport. Removing openness may leave the recipe open or select another valid neighborhood route; it must not be labeled a mathematical refutation.

Fourth, connect synthesis and a nonalgebraic construction. Register actual denotation theorems for a small Leant provider family, and implement quotient descent or another universal-property workload. This tests whether the design generalizes beyond arithmetic side conditions.

Finally, evaluate the interface. Only after the shared backend works should the full mathematical surface and reader study be treated as language experiments. The examples examined in this article are development examples, not held-out benchmarks. Reserve previously unexamined theorem families for evaluation.

17.3 Adversarial acceptance matrix

Table 5: Priority end-to-end tests. The ground companion exercises only the explicitly described symbolic subset.

Id	Mutation	Required outcome
J1	Replace neighborhood equality by equality at a point.	Derivative recipe stays open; no inferred germ.
J2	Specialize the induction hypothesis before the successor step.	Request the missing set-wide or neighborhood assertion.
J3	Delete a Frey divisibility premise.	Reject quotient transport or expose its exact residual.
J4	Use nonzero as a synonym for unit in a general ring.	Request the invertibility contract; do not silently change rings.
J5	Change the order, topology, measure, or scalar action.	Report statement change; recheck affected evidence.
J6	Reuse a branch-local proof in a sibling.	Reject context reuse.
J7	Solve a guard using the result whose recipe needs that guard.	Reject cyclic derivation without a separate well-founded proof.
J8	Omit one alternative but mark the frontier complete.	Closure-certificate validation fails.
J9	Stop enumeration at a budget limit.	Retained proofs stay usable; coverage is explicitly incomplete.
J10	Display an unproved assertion unused by the terminal theorem.	Publication as a fully verified narrative fails.
J11	Substitute a formal coefficient theorem for an analytic identity.	Require the semantic analytic bridge.
J12	Treat a finite jet as an exact identity.	Preserve precision; reject stronger consumers.
J13	Refute a list contract using an unrealizable positive length.	Require a concrete admissible input or realization theorem.
J14	Replace one provider by another with the same printed short name.	Recheck exact provider identity and denotation theorem.
J15	Combine factorization and continuity of different chosen factors.	Dependent telescope rejects the mismatched witnesses.
J16	Use an arbitrary proof with a requested recipe’s label.	Replay the recipe or report the method mismatch.

17.4 Answers to the syntheses’ remaining questions

The first synthesis asks what artifact and evidence would advance the campaign; the second lists ten next questions about costs, triggering, simplification, abstraction, domains, structure selection, rule generation, reading, and independent implementation. Juniper’s responses are concrete but provisional.

The next artifact is a real typed operation with a validated finite frontier, not another full grammar. The grounding policy is explicit in Section 7; its usefulness remains to be measured. Property inference fires at demands and registered recipe boundaries, not at every possible theorem in the environment. Simplification is handled by conversion or a recorded transport, and initial edits trigger rebuilding rather than unverified persistent reuse.

The abstraction criterion is consumer-specific: positive soundness, decision completeness on the admissible image, and negative realization are separate. Almost-everywhere evidence is tested with an integral consumer, not an unrestricted pointwise consumer. Lexical structure selection is proposed as a way to stabilize meaning, not assumed to shorten every generated preamble.

The cost test uses identical library and service baselines. The reading test asks users to reconstruct quantifiers, domains, and remaining obligations, not merely to prefer a prettier page. Independent implementations should agree on finite-manifest semantics and certificate acceptance, even when they choose different source syntax or proof routes. The included exhaustive oracle is an initial finite control, not a substitute for that independent implementation exercise.

The stop rule is substantive: if the property layer does not improve the strongest Lean-plus-library-plus-renderer baseline, Juniper should remain a library and tooling profile. A kernel modification is justified only by a measured obstruction that survives these less invasive alternatives.

18 Conclusion

Juniper’s intended concision comes from *naming a mathematical move and reconstructing its licensed evidence*, not from hiding an arbitrary proof script. Properties and behavioral constraints become usable typing information, but the objects, structures, domains, and meanings to which they refer remain fixed.

The fourth-round increment is a precise treatment of incomplete reasoning. Residual telescopes preserve dependencies. Finite antichain frontiers expose alternative sufficient premise sets. Derivation graphs certify each route, and a closure condition can certify completeness relative to the finite rule manifest. The negative cases remain intelligible: missing evidence, unsupported rules, unrealizable counterexamples, and resource exhaustion are different outcomes.

The inverse-derivative example shows the desired mathematical workflow particularly clearly. The author supplies the branch and the recurrence; the analytic library licenses differentiation; a symbolic backend proves the polynomial identities; the property system propagates nonvanishing and domain information. Each component contributes a different kind of evidence, and none is allowed to impersonate the others.

The resulting proposal is implementable incrementally without changing Lean’s kernel. The delivered executable fragment tests a meaningful part of that design, while its limitations are explicit. The decisive next result would be an end-to-end operation on a real Lean theorem, preserving its statement and improving authoring or repair against a strong controlled baseline.

A Reproduction and artifact contents

The archive contains this complete LaTeX source and its compiled PDF, plus a standard-library-only Python companion. No third-party fonts or copied repository source files are required to rebuild the document.

```

Juniper/
  Juniper.tex
  Juniper.pdf
  README.md
  build.sh
  source-manifest.json
  companion/
    juniper_frontier.py
    test_frontier.py
    inverse_polynomials.py
    example.jnp
    example-result.json
    Generated.lean
    test-results.json
    test-run.log
    inverse-results.json
    inverse-run.log

```

Run the executable checks from the companion directory:

```

python test_frontier.py
python inverse_polynomials.py
python juniper_frontier.py example.jnp \
  --json example-result.json --lean Generated.lean

```

The run recorded here used Python 3.13.5. The code uses only the standard library and syntax available in Python 3.10 and later. The frontier program’s optional `-max-attempts` bounds rule-support combinations; it is not a wall-clock deadline or a complete memory bound.

With a Lean installation, `lean Generated.lean` is the intended check for the generic export; no Mathlib import is needed. This command was not executed in the recorded environment. Compiling it would validate generic implication assembly, not the unimplemented mathematical reifier or the frontier completeness theorem.

For the article, use `latexmk -pdf Juniper.tex` or run `pdflatex` sufficiently many times to resolve references and the table of contents. The build script performs the former. Intermediate build files are not part of the release archive.

B Further inverse-derivative polynomial output

For completeness, the exact reference computes

$$\begin{aligned}
 P_5 &= -u_1^3 u_5 + 15u_1^2 u_2 u_4 + 10u_1^2 u_3^2 - 105u_1 u_2^2 u_3 + 105u_2^4, \\
 P_6 &= -u_1^4 u_6 + 21u_1^3 u_2 u_5 + 35u_1^3 u_3 u_4 - 210u_1^2 u_2^2 u_4 \\
 &\quad - 280u_1^2 u_2 u_3^2 + 1260u_1 u_2^3 u_3 - 945u_2^5.
 \end{aligned}$$

The article’s recurrence and analytic theorem explain how these expressions are to be interpreted. The JSON output reports them as exact Python reference results, not as kernel-checked identities about a particular function. Their degree invariants provide additional checks but do not replace equality certification.

C Source scope and attribution

The review date is September 6, 2026. Both primary round-three synthesis bodies were read, with selected appendix material and a direct reading of Trellis’s finite-qualifier and property-typing sections. The individual proposals are credited through the syntheses except where Trellis is cited directly. This is not a claim to have reread every file from all prior rounds.

Direct repository observations use the following immutable snapshots:

Repository	Revision
Brainstorm	bf3333905995060870f8585e297ffc16f3fe7e86
ProveIt	24ce8bd743eaab64a91ce90725ea00f498d319d2
anthropics/fermats-last-theorem	aa2d8b34692b16c70f699536de0d8e75b9a3e9ef
Leant	3a40904be8a410d832d9ab6900b3d3e7b425eccb

The source manifest additionally records blob hashes for the directly examined files. The original mathematical and software work belongs to its respective authors and repository contributors. The FLT Frey-package definition credits the Imperial College London formalization and its named authors. Juniper is a design proposal informed by those sources, not a claim to authorship of their results.

References

- [1] Brainstorm campaign. *From Properties to Proof Obligations: Round 3 Synthesis*. September 6, 2026. <https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/synthesis/unified-report.tex>.
- [2] Brainstorm campaign. *Nine Refinements: Property-Aware Typing in a Mathematician’s Proof Language over Lean*. September 6, 2026. Main report and appendix. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/unified_report/unified_report.tex.
- [3] Brainstorm campaign. *Trellis*, round-three proposal, especially property typing, finite qualifier closure, structure selection, and behavioral contracts. <https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/ideas/trellis/trellis.tex>.
- [4] ProveIt contributors. *AutonomousIteratedDeriv.lean*. Declarations `iteratedDeriv_eq_comp` and `iteratedDeriv_eq_comp_of_differentiable`. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [5] ProveIt contributors. *InverseDerivativeRecursion.lean*. Formal/analytic distinction, local inverse derivative, and iterated operator theorem. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/InverseDerivativeRecursion.lean>.
- [6] FLT repository contributors. *Def_FLTPrelim_FreyPackage.lean*. Credits Kevin Buzzard, Ruben Van de Velde, Pietro Monticone, and the Imperial College London FLT formalization. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/Definitions/Def_FLTPrelim_FreyPackage.lean.
- [7] FLT repository contributors. *S_FreyPackage_freyCurveInt_map.lean*. Integer-model map and accompanying arithmetic lemmas. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_freyCurveInt_map.lean.

- [8] Leant contributors. `src/Leant/Synth/Verification.hs`. Callback-acceptance receipts and candidate verification.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs>.
- [9] Leant contributors. `src/Leant/Synth/PostVerification.hs`. Generative epochs and exact bounded receipt permutations.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/PostVerification.hs>.
- [10] Leant contributors. `src/Leant/Synth/Length/Contract.hs`. Passive provider and scalar/product behavioral contracts.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Length/Contract.hs>.
- [11] Lean language reference. *Subtypes*. Consulted September 6, 2026.
<https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/>.
- [12] Lean language reference. *Inductive Types*, run-time representation and zero-overhead subtypes. Consulted September 6, 2026.
<https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.
- [13] Lean language reference. *Holes*, synthetic opaque and delayed assigned metavariables. Consulted September 6, 2026.
<https://lean-lang.org/doc/reference/latest/Terms/Holes/>.
- [14] Lean language reference. *Elaboration and Compilation*. Consulted September 6, 2026.
<https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>.
- [15] Mathlib documentation. *Mathlib.Tactic.FunProp*. Compositional properties, morphisms, and transition theorem search. Consulted September 6, 2026.
https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.
- [16] Lean language reference. *The grind tactic*. Consulted September 6, 2026.
<https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [17] Lean language reference. *The mvcgen tactic: Overview*. Consulted September 6, 2026.
<https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/>.
- [18] Jana Dunfield and Neel Krishnaswami. *Bidirectional Typing*. ACM Computing Surveys 54(5), 2021. DOI: 10.1145/3450952. Author’s paper page:
<https://research.cs.queensu.ca/home/jana/papers/bidir-survey/>.
- [19] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016. DOI: 10.1145/2908080.2908093.
<https://arxiv.org/abs/1510.08419>.
- [20] Danel Ahman and coauthors. *Dijkstra Monads for Free*. POPL 2017. Author project page and abstract:
<https://fstar-lang.org/papers/dm4free/>.
- [21] Mario Carneiro. *Lean4Lean: Verifying a Typechecker for Lean, in Lean*. arXiv:2403.14064, revised 2025.
<https://arxiv.org/abs/2403.14064>.